

Dokumentacja Biblioteki PUTM EV CAN

Michał Błotniak

3 lutego 2026

Streszczenie

Niniejszy dokument stanowi instrukcję obsługi biblioteki `PUTM_EV_CAN_LIBRARY`. Biblioteka została zaprojektowana dla systemów wbudowanych (STM32) oraz środowiska ROS2. Główne cechy to: statyczna alokacja pamięci, obsługa wielu rodzin STM32 (F4, G4, L4, H7) dzięki warstwie abstrakcji HAL, odbiór danych oparty na przerwaniach (Interrupt-Driven) oraz wbudowana diagnostyka LED.

Spis treści

1	Wymagania systemowe i zależności	2
1.1	Instalacja zależności Python	2
2	Integracja i Konfiguracja w CMake	3
2.1	Dodawanie biblioteki do projektu	3
2.2	Wybór konfiguracji (STM32 vs ROS2)	3
2.2.1	Konfiguracja dla STM32	3
2.2.2	Konfiguracja dla ROS2 / Linux	3
2.3	Włączenie obsługi CAN FD	3
3	Konwencja Nomenklatury Magistral	4
4	Poprawna inicjalizacja	4
4.1	Krok 1: Instancje Globalne	4
4.2	Krok 2: Wywołanie Init	4
5	Diagnostyka LED	4
5.1	Konfiguracja Diody	5
5.2	Znaczenie Sygnałów (Mruganie)	5
6	Wysyłanie danych (TX)	6
7	Odbieranie danych (RX)	7
7.1	Krok 1: Rejestracja Callbacku	7
7.2	Krok 2: Polling w pętli głównej	7
8	Najczęściej popełniane błędy	7
9	Szczegółowa Architektura Systemu	8
9.1	Warstwa 1: Fasada Aplikacji (<code>app_can_lib</code>)	8
9.2	Warstwa 2: Logika i Routing	8
9.2.1	Interfejs (<code>can_interface</code>)	8
9.2.2	Router (<code>message_handler</code>)	8
9.3	Warstwa 3: Abstrakcja Sprzętowa HAL (<code>can_hal</code>)	8
9.4	Warstwa 4: Definicje Danych (<code>dbc/generated</code>)	8

1 Wymagania systemowe i zależności

Prawidłowa kompilacja i działanie biblioteki PUTM_EV_CAN_LIBRARY wymaga zapewnienia odpowiedniego środowiska programistycznego. Poniższa tabela przedstawia minimalne wersje narzędzi oraz bibliotek wykorzystywanych w projekcie.

Narzędzie / Biblioteka	Wersja Minimalna	Uwagi
CMake	3.16	Wymagane przez konfigurację projektu.
Standard C++	C++20	Wymuszone w pliku <code>CMakeLists.txt</code> .
Standard C	C11	Standard dla kodu C.
Python	3.8+	Niezbędny do uruchomienia skryptów generujących kod z DBC.
Kompilator C++ (ARM)	GCC $\geq$ 10.0	Musi obsługiwać flagi standardu C++20 (np. <code>arm-none-eabi-gcc</code>).
Biblioteka HAL	STM32XXxx HAL	Zależność sprzętowa dla mikrokontrolerów różnych serii.
Python: <code>cantools</code>	40.7.1	Generator kodu C z plików DBC.
Python: <code>python-can</code>	(auto)	Instalowana automatycznie jako zależność <code>cantools</code> .

Tabela 1: Wymagania systemowe projektu

1.1 Instalacja zależności Python

Skrypt generujący kod (`dbc/setup.py`) jest uruchamiany automatycznie przez CMake i wymaga zainstalowanych bibliotek w środowisku Pythona. Biblioteka `cantools` w wersji 40.7.1 pociąga za sobą wymagane zależności (m.in. `python-can`, `bitstruct`).

Aby zainstalować wymagane pakiety, zaleca się użycie poniższego polecenia w terminalu:

```
1 pip install cantools==40.7.1
```

Listing 1: Instalacja wymaganych bibliotek Python

Brak tych bibliotek spowoduje błąd konfiguracji CMake na etapie **Checking and regenerating DBC files**.

2 Integracja i Konfiguracja w CMake

Biblioteka jest dystrybuowana jako moduł CMake. Aby użyć jej w projekcie, należy zmodyfikować główny plik `CMakeLists.txt` projektu.

2.1 Dodawanie biblioteki do projektu

W głównym pliku `CMakeLists.txt` (np. w katalogu głównym projektu STM32 lub węzła ROS2) należy dodać podkatalog z biblioteką oraz zlinkować ją z plikiem wykonywalnym.

```
1 # 1. Dodanie podkatalogu biblioteki
2 add_subdirectory(Core/Inc/PUTM_EV_CAN_LIBRARY)
3
4 # 2. Linkowanie biblioteki z twoim projektem
5 # 'twoj_projekt' to nazwa zdefiniowana w add_executable()
6 target_link_libraries(twoj_projekt PRIVATE putm_ev_can)
```

Listing 2: Podstawowa konfiguracja CMake

2.2 Wybór konfiguracji (STM32 vs ROS2)

Biblioteka wspiera dwa backhendy sprzętowe. Wybór następuje poprzez ustawienie zmiennej `PUTM_BACKEND` przed wywołaniem `add_subdirectory`.

2.2.1 Konfiguracja dla STM32

Jest to domyślna konfiguracja. Wymaga ona, aby w projekcie dostępna była biblioteka HAL.

```
1 set(PUTM_BACKEND "STM32" CACHE STRING "Select backend" FORCE)
2 add_subdirectory(Core/Inc/PUTM_EV_CAN_LIBRARY)
```

Listing 3: Konfiguracja dla STM32

2.2.2 Konfiguracja dla ROS2 / Linux

W tym trybie biblioteka korzysta z systemowego SocketCAN.

```
1 set(PUTM_BACKEND "LINUX" CACHE STRING "Select backend" FORCE)
2 add_subdirectory(lib/PUTM_EV_CAN_LIBRARY)
```

Listing 4: Konfiguracja dla Linux/ROS2

2.3 Włączenie obsługi CAN FD

Domyślnie biblioteka pracuje w trybie Classic CAN. Aby włączyć obsługę ramek do 64 bajtów:

```
1 option(PUTM_USE_CAN_FD "Enable CAN FD" ON)
```

3 Konwencja Nomenklatury Magistral

W projekcie pojazdu zdefiniowano trzy fizyczne magistrale CAN. Aby zachować spójność w kodzie (na STM32 i w ROS2), **należy bezwzględnie** stosować poniższe nazewnictwo instancji klasy AppCAN.

Nazwa Instancji	Funkcja Magistrali	Opis
can_m	Main	Główna magistrala, większość danych, telemetria.
can_pt	Powertrain (ECU/INV)	Komunikacja wyłącznie falownik - silniki.
can_dv	Driverless	Wymiana sygnałów w trybie jazdy autonomicznej.

Tabela 2: Obowiązująca nomenklatura instancji AppCAN

Wszystkie przykłady w dalszej części dokumentacji będą odwoływać się do tych konkretnych nazw zmiennych.

4 Poprawna inicjalizacja

Biblioteka nie używa dynamicznej alokacji pamięci. Instancje muszą być zadeklarowane jako zmienne globalne.

4.1 Krok 1: Instancje Globalne

W pliku main.cpp (lub odpowiedniku w ROS2) należy utworzyć obiekty dla wykorzystywanych magistral.

```
1 /* Private variables */
2 // Definicja instancji zgodnie z przyjętą nomenklaturą
3 AppCAN can_m;    // Main Bus
4 AppCAN can_pt;   // Powertrain Bus
5 AppCAN can_dv;   // Driverless Bus
```

4.2 Krok 2: Wywołanie Init

Inicjalizację należy przeprowadzić wewnątrz funkcji main(), po konfiguracji peryferiów przez CubeMX. Każdej instancji przypisujemy odpowiedni uchwyt sprzętowy (hfdcanX lub hcanX).

```
1 /* USER CODE BEGIN 2 */
2
3 // 1. Inicjalizacja Main CAN (np. FDCAN1)
4 if (!can_m.Init(&hfdcan1)) {
5     Error_Handler(); // Błąd inicjalizacji Main
6 }
7
8 // 2. Inicjalizacja Powertrain CAN (np. FDCAN2)
9 if (!can_pt.Init(&hfdcan2)) {
10    Error_Handler(); // Błąd inicjalizacji PT
11 }
12 /* USER CODE END 2 */
```

Uwaga: Biblioteka automatycznie wykrywa rodzinę procesora (np. G4 vs F4) i dostosowuje typ uchwytu (FDCAN vs bxCAN).

5 Diagnostyka LED

Biblioteka posiada wbudowany mechanizm automatycznej sygnalizacji stanu magistrali za pomocą diody LED. Funkcjonalność ta pozwala na szybką diagnostykę problemów (np. odpięta wtyczka, uszkodzona magistrala, błędy ramek) bez konieczności podłączania debuggera.

5.1 Konfiguracja Diody

Dla każdej instancji AppCAN (np. `can_m`, `can_pt`) można przypisać osobną diodę. Konfigurację należy wykonać **raz**, zaraz po pomyślnej inicjalizacji biblioteki.

```
1 if (can_m.Init(&hfdcan1)) {  
2     // Przypisanie diody (np. LED1) do tej magistrali  
3     can_m.ConfigStatusLed(LED1_GPIO_Port, LED1_Pin);  
4 }
```

5.2 Znaczenie Sygnałów (Mruganie)

Po skonfigurowaniu, biblioteka automatycznie steruje diodą wewnątrz metody `Poll()`. Użytkownik nie musi (i nie powinien) ręcznie sterować tymi pinami.

Zachowanie Diody	Stan Magistrali	Opis
Mruganie 1 Hz (500ms ON, 500ms OFF)	OK	Magistrala działa poprawnie (Error Active). Komunikacja jest możliwa.
Nierówne Mruganie (Podwójny błysk co 1s)	WARNING	Wykryto błędy transmisji (Error Passive / Warning). Może oznaczać zakłócenia lub błędy ramek.
Świeci Ciągłe (Solid ON)	BUS OFF	Krytyczny błąd magistrali. Kontroler CAN odłączył się od sieci (np. zwarcie).

Tabela 3: Kody diagnostyczne diody statusowej

6 Wysyłanie danych (TX)

Wysyłanie odbywa się poprzez metodę **Send** wywołaną na odpowiedniej instancji magistrali (zazwyczaj **can_m**). Pamiętaj o używaniu poprawnego prefiksu **PUTM_CAN_M_** (lub innego, zależnie od pliku DBC).

```
1 // Przykład w petli głównej (co 100ms)
2 if (HAL_GetTick() - last_tx > 100) {
3     last_tx = HAL_GetTick();
4
5     // 1. Tworzenie i zerowanie struktury
6     PUTM_CAN_M_pc_main_data_t tx_msg = {0};
7
8     // 2. Wypełnianie danych
9     tx_msg.vehicle_speed = 125;
10    tx_msg.inverters_ready = 1;
11
12    // 3. Wysłanie na magistrale MAIN (can_m)
13    if (can_m.Send(PUTM_CAN_M_PC_MAIN_DATA_FRAME_ID, tx_msg)) {
14        // Opcjonalnie: dodatkowa logika po wysłaniu
15    }
16 }
```

7 Odbieranie danych (RX)

Odbiór w nowej wersji biblioteki działa w trybie **Interrupt-Driven (Przerwania)**. Oznacza to, że dane są pobierane natychmiast po ich nadejściu, a nie w pętli głównej.

7.1 Krok 1: Rejestracja Callbacku

Rejestrację wykonujemy raz, przed pętlą główną. **OSTRZEŻENIE:** Funkcja lambda (callback) jest wywoływana w kontekście przerwania sprzętowego (ISR). Należy w niej unikać:

- Funkcji blokujących (HAL_Delay).
- Długich operacji matematycznych.
- Funkcji wejścia/wyjścia (np. printf).

Zaleca się jedynie ustawianie flag, zmiennych volatile lub szybkie sterowanie pinami.

```
1 // Odbiór danych z Main CAN
2 can_m.RegisterCallback<PUTM_CAN_M_rearbox_safety_t>(
3     PUTM_CAN_M_REARBOX_SAFETY_FRAME_ID,
4     [](const PUTM_CAN_M_rearbox_safety_t& msg) {
5
6         // Ta funkcja wykonuje się w PRZERWANIU!
7         // Przykład bezpiecznej operacji: sterowanie LED
8         if (msg.safety_tsmp == 1) {
9             HAL_GPIO_WritePin(LED3_GPIO_Port, LED3_Pin, GPIO_PIN_RESET);
10        } else {
11            HAL_GPIO_WritePin(LED3_GPIO_Port, LED3_Pin, GPIO_PIN_SET);
12        }
13    }
14 );
```

7.2 Krok 2: Polling w pętli głównej

Mimo że odbiór danych jest asynchroniczny, metoda Poll() nadal musi być wywoływana. Jej zadaniem jest teraz obsługa **diody diagnostycznej**.

```
1 while (1) {
2     // Obsługa mrugania dioda diagnostyczna
3     // (Odbiór ramek dzieje się automatycznie w tle)
4     can_m.Poll();
5     can_pt.Poll();
6     can_dv.Poll();
7
8     // ... reszta kodu ...
9 }
```

8 Najczęściej popełniane błędy

- **Użycie złego prefiksu:** Biblioteka teraz używa PUTM_CAN_M_ (dla Main DBC). Użycie starego PUTM_CAN_1_ spowoduje błąd kompilacji.
- **Brak słowa volatile:** Jeśli zmienna jest modyfikowana w callbacku (RX) i czytana w main(), musi być oznaczona jako volatile, inaczej kompilator może ją źle zoptymalizować.
- **Pomylenie instancji:** Wysłanie wiadomości na can_m zamiast can_pt, przez co falownik jej nie otrzyma.
- **Brak wywołania Poll():** Skutkuje brakiem sygnalizacji LED (dioda nie będzie mrugać mimo poprawnej pracy).

9 Szczegółowa Architektura Systemu

Biblioteka została zaprojektowana w oparciu o wzorzec projektowy *Fasada* (Facade) oraz *Wstrzykiwanie Zależności*. Architektura jest podzielona na cztery główne warstwy.

Rysunek 1: Diagram warstwowy biblioteki

9.1 Warstwa 1: Fasada Aplikacji (app_can_lib)

Jest to jedyna warstwa, z którą bezpośrednio intereaguje użytkownik końcowy.

- **Odpowiedzialność:** Ukrywa złożoność wewnętrzną, zarządza czasem życia obiektów i obsługuje diagnostykę LED w metodzie `Poll()`.

9.2 Warstwa 2: Logika i Routing

9.2.1 Interfejs (can_interface)

Rejestruje callback odbiorczy w warstwie HAL. Gdy przyjdzie przerwanie, HAL przekazuje surowe dane tutaj, a interfejs pakuje je w struktury C++ (korzystając z DBC).

9.2.2 Router (message_handler)

Przechowuje listę callbacków użytkownika. Zapewnia bezpieczeństwo wątkowe przy rejestracji nowych funkcji.

9.3 Warstwa 3: Abstrakcja Sprzętowa HAL (can_hal)

Warstwa izolująca kod od sprzętu. Została znacznie rozbudowana w nowej wersji.

- **Selektor (stm32_hal_selector.hpp):** Automatycznie wykrywa rodzinę procesora (np. G484xx) i dołącza odpowiednie nagłówki HAL. Definiuje typ `CanHandleType` (FDCAN dla G4/H7, bxCAN dla F4/L4).
- **Dispatcher Przerwań:** Klasa `Stm32CanHal` posiada statyczną tablicę, która mapuje uchwyt sprzętowe na obiekty C++. Pozwala to globalnej funkcji przerwania C (`HAL_FDCAN_RxFifo0Callback`) odnaleźć właściwą instancję biblioteki i przekazać jej dane.

9.4 Warstwa 4: Definicje Danych (dbc/generated)

Kod generowany automatycznie przez skrypt Python na podstawie plików `.dbc`. Zawiera definicje struktur (np. `PUTM_CAN_M_bms_lv_main_t`) oraz funkcje pakujące.